

CI/CD Observability in Containerized Environments

Eduardo Sousa

**A dissertation preparation for the degree of
of Master of Science, Specialisation Area of Cybersecurity And
Systems Administration**

**Advisor: Dr. Alexandre Bragança
Supervisor: Dr. Joao Silva**

Statement of Integrity

I hereby declare having conducted this academic work with integrity.

I have not plagiarised or applied any form of undue use of information or falsification of results along the process leading to its elaboration.

Therefore the work presented in this document is original and authored by me, having not previously been used for any other end.

I further declare that I have fully acknowledged the Code of Ethical Conduct of P.PORTO.

ISEP, Porto, December 31, 2025

Abstract

Internal CI/CD services (GitLab, Jenkins, Nexus, SonarQube, Dependency-Track) often run in Docker on mixed Linux/Windows hosts without administrative access. In these constrained environments, teams struggle to detect faults quickly and to diagnose root causes because telemetry is fragmented and host-level instrumentation is not permitted. This work proposes a container-first observability architecture that collects metrics, logs and traces from non-privileged containers using sidecar exporters, agent collectors and runtime APIs, and correlates signals in a unified metrics pipeline with dashboards and alerts. The approach is validated on a four-server, on-premise setup and evaluated with MTTD, MTTR, pipeline throughput, alert precision and overhead (target below 5%). The expected contribution is a reference architecture, reproducible prototype and decision guidance that improve CI/CD reliability under strict privilege constraints. **Keywords:** CI/CD, Observability, Containers,

Tracing, Logging, Monitoring

Resumo

Trabalhos escritos em língua Inglesa devem incluir um resumo alargado com cerca de 1000 palavras, ou duas páginas.

Se o trabalho estiver escrito em Português, este resumo deveria ser em língua Inglesa, com cerca de 200 palavras, ou uma página.

Para alterar a língua basta ir às configurações do documento no ficheiro `main.tex` e alterar para a língua desejada ('english' ou 'portuguese')¹. Isto fará com que os cabeçalhos incluídos no template sejam traduzidos para a respetiva língua.

¹Alterar a língua requer apagar alguns ficheiros temporários; O target **clean** do **Makefile** incluído pode ser utilizado para este propósito.

Contents

List of Figures	xi
List of Tables	xiii
List of Symbols	xv
1 Introduction	1
1.1 Context and Motivation	1
1.2 Problem Statement	1
1.2.1 Extended abstract summary	1
1.2.2 Contributions	2
1.3 Research Objectives	2
1.4 Research Questions	2
1.5 Methodology Overview	3
1.6 Ethical Considerations	3
1.7 Structure of the Dissertation	4
2 Skill Management and Project Planning	5
2.1 Skill Management	5
2.1.1 Skills Identification	5
2.1.2 Skills Assessment and Development	5
2.1.3 Strategy to Improve Skills	5
2.2 Project Management	6
2.2.1 Phase 1 – PREPD deliverable (November 2025–January 2026)	6
2.2.2 Work Plan (extended abstract)	6
2.2.3 Phase 2 – Thesis development (January–June 2026)	7
2.2.4 Project Charter, Scope and Schedule	7
2.2.5 Monitoring and Controlling Procedures	7
2.2.6 Risk Identification and Management	7
3 Literature review	9
3.1 PRISMA Methodology Overview	9
3.1.1 Identification	10
3.1.2 Screening	10
3.1.3 Eligibility	10
3.1.4 Inclusion and Data Extraction	10
3.2 Prisma Flow Diagram	10
3.2.1 Quality assessment criteria	10
3.3 Related Work on Observability in CICD Environments	11
3.3.1 Representative studies	11
3.3.2 Preliminary findings from the extended abstract	11

3.3.3	Observability frameworks and telemetry collection	11
3.3.4	Correlation mechanisms	12
3.3.5	Performance metrics and evaluation methods	12
3.4	Gaps in Existing Research	12
3.5	Summary	12
3.6	Answering the Research Question	12
4	Conclusion and Next Steps	13
4.0.1	Validation targets from the extended abstract	14
	Bibliography	15
	Appendix A Appendix Title Here	17

List of Figures

List of Tables

List of Symbols

a	distance	m
P	power	W (Js^{-1})
ω	angular frequency	rad

Chapter 1

Introduction

1.1 Context and Motivation

Internal continuous integration and continuous delivery (Continuous Integration / Continuous Delivery (CI/CD)) services such as GitLab, Jenkins, SonarQube, Nexus and DependencyTrack are central components of many software delivery pipelines. Failures or degraded performance in these services reduce development velocity and can delay releases. This dissertation examines practical observability approaches for containerized CI/CD deployments operating under restricted administrative privileges and in heterogeneous operating environments (mixed Linux and Windows hosts).

The motivation for focusing on constrained, nonprivileged deployments arises from repeated operational observations in industry settings where teams manage their own onpremise infrastructure but lack host administrative access. In such contexts, conventional cloudnative approaches — for example, using privileged host exporters or privileged DaemonSets in Kubernetes — may be unavailable. The research therefore prioritises deployment patterns and tooling that can be applied without modifying service images or requiring host admin rights.

1.2 Problem Statement

The specific research problem addressed in this work is: how to design and implement an integrated observability framework for containerized CI/CD platforms operating under nonprivileged access constraints and heterogeneous infrastructure, such that the resulting artifact supports reproducible evaluation of reliability and incident response improvements.

1.2.1 Extended abstract summary

Internal CI/CD services (GitLab, Jenkins, Nexus, SonarQube, DependencyTrack) often run in Docker containers on mixed Linux/Windows hosts without administrative access. In these constrained environments, teams commonly face difficulty detecting faults promptly and diagnosing root causes because telemetry is fragmented and hostlevel instrumentation is not permitted. This dissertation proposes a containerfirst observability architecture that collects metrics, logs and traces from nonprivileged containers using sidecar exporters, agent collectors and runtime APIs, and correlates signals in a unified telemetry pipeline with dashboards and alerts. The approach will be validated on a fourserver, onpremise deployment (three Linux, one Windows) and evaluated using Mean Time To Detection (MTTD), Mean Time To Recovery (MTTR), pipeline throughput, alert precision and instrumentation overhead

(operational target: below 5%). The expected contribution is a documented reference architecture, a reproducible prototype and decision guidance to improve CI/CD reliability under strict privilege constraints.

1.2.2 Contributions

The primary contributions expected from this research are:

- A containerfirst reference architecture for collecting metrics, logs and traces from nonprivileged containers in mixed OS onpremise environments.
- A reproducible prototype (container images and deployment templates) demonstrating feasible instrumentation patterns (sidecars, collectors, log forwarding) that do not require host administrative privileges.
- An empirical evaluation reporting measured changes in MTTD, MTTR, pipeline throughput and measured instrumentation overhead, including a documented measurement methodology to support reproducibility.
- Practical decision guidance and configuration artefacts to support adoption by operations teams that cannot modify service images or obtain host administrative access.

1.3 Research Objectives

The research objectives are:

- **O1 Reference architecture:** Define a containerfirst reference architecture specifying exporters, collectors, storage backends and visualization components deployable without host administrative privileges.
- **O2 Prototype implementation:** Produce a reproducible prototype (container images and deployment templates) that instruments representative CI/CD services in the target environment.
- **O3 Quantitative evaluation:** Define and execute an empirical evaluation comparing MTTD, MTTR, telemetry pipeline throughput and instrumentation overhead before and after deployment, with documented measurement procedures.
- **O4 Performance constraints:** Characterize resource overhead and document trade-offs; the evaluation includes targets and thresholds (for example, an operational target of under 5% resource overhead) to guide assessment.
- **O5 Reproducibility and guidance:** Provide configuration templates, deployment scripts and adoption guidance for constrained, onpremise environments.

Each objective is scoped to be actionable and measurable. For example, Objective O3 includes explicit measurement procedures (baseline collection, controlled fault injection and paired comparisons) to enable reproducible evaluation rather than solely qualitative assessment.

1.4 Research Questions

The dissertation is guided by the following questions:

1. How can metrics, logs and distributed traces be collected from containerized CI/CD services when only nonprivileged containers are available (no host administrative access)?
2. Which instrumentation and deployment patterns (sidecars, agent collectors, push vs pull pipelines) best support crossserver correlation in mixed OS environments while maintaining acceptable resource overhead?
3. What are the practical constraints and tradeoffs (data fidelity, retention, network and CPU overhead, security) introduced by nonprivileged telemetry collection, and how can they be mitigated in a reproducible reference architecture?

1.5 Methodology Overview

This research follows a design science approach [1], combining a systematic literature review, comparative technology assessment, artifact design, prototype implementation and empirical evaluation. The work proceeds in five phases:

Phase 1 – Problem Analysis and Literature Review: A PRISMA-guided systematic review of observability frameworks, containerized monitoring and CI/CD reliability to identify approaches and gaps relevant to constrained environments.

Phase 2 – Technology Assessment: Evaluate candidate tools (for example, Prometheus [2], Loki, Jaeger, OpenTelemetry) against criteria including nonprivileged execution, cross-platform support and resource overhead.

Phase 3 – Architecture Design: Specify component placement (exporters, collectors, storage and visualization), data flows and correlation mechanisms, producing deployment diagrams and configuration templates.

Phase 4 – Prototype Implementation: Implement and document a reproducible prototype using sidecars and collector containers where appropriate; provide deployment automation and configuration artifacts.

Phase 5 – Evaluation and Validation: Quantitatively evaluate MTTD, MTTR and instrumentation overhead; perform controlled fault injection experiments and collect qualitative feedback from practitioners.

The methodological choices reflect the dual goals of creating a practical artefact (the reference architecture and prototype) and generating evaluative evidence about its efficacy. Quantitative evaluation is complemented by qualitative data from practitioner interviews to capture operational acceptability and deployment constraints that are difficult to reproduce in laboratory conditions.

1.6 Ethical Considerations

Human subjects activity (structured interviews) and access to operational telemetry will follow approved protocols. Participants provide informed consent; data are anonymized and handled according to institutional and organizational policies. Production deployments will be preceded by staging tests and changemanagement procedures to avoid operational disruption.

The study will also observe data minimisation principles for telemetry collection and will avoid collecting personal data unless explicitly required and justified; any such collection will follow institutional review procedures and applicable regulations. When presenting empirical results, host identifiers and other sensitive configuration details will be anonymised to protect organisational confidentiality.

1.7 Structure of the Dissertation

The remainder of the dissertation is organised as follows: Chapter 2 presents project planning and skill management; Chapter 3 describes the systematic literature review and related work; Chapter 4 summarises conclusions and outlines next steps for prototype implementation and evaluation. Appendices provide supplementary materials and configuration artifacts.

Chapter 2

Skill Management and Project Planning

2.1 Skill Management

This section summarises the skills required for the research activities and the planned approach for acquiring or consolidating them.

2.1.1 Skills Identification

Key technical skills include container orchestration and debugging, telemetry and observability tooling (OpenTelemetry, Prometheus, Loki, Jaeger), scripting for automation, and experimental design for empirical evaluation. Soft skills include project management and structured interviewing for qualitative validation.

The selection emphasises skills that are directly relevant to the prototype implementation and evaluation: reliable collection of telemetry from containers without elevated privileges, crossplatform troubleshooting (Linux and Windows containers), and reproducible deployment using containers and scripts. Competence in interpreting timeseries data and traces for incident diagnosis is also required to validate the proposed evaluation metrics.

2.1.2 Skills Assessment and Development

An initial self-assessment identifies gaps in advanced tracing and Windows container telemetry; these will be addressed through focused practical exercises, small proof-of-concepts, and targeted readings during Phase 1.

2.1.3 Strategy to Improve Skills

The strategy combines short, time-boxed experiments, community documentation and tutorials, and incremental integration into the prototype. All skill development tasks are recorded and linked to project milestones to ensure traceability.

Planned activities include the following practical exercises:

- Implement a minimal sidecar exporter for a simple containerised service to validate nonprivileged metrics collection.
- Configure Prometheus scraping against nonstandard endpoints and verify collection reliability on both Linux and Windows hosts.

- Perform lightweight tracing with OpenTelemetry to confirm context propagation using proxy or sidecar approaches.

Each exercise has acceptance criteria (e.g. successful metric collection for 24 hours under synthetic load) to provide objective evidence of skill acquisition.

2.2 Project Management

The research timeline comprises two main phases: the PREPD deliverable (November 2025–January 2026) and thesis development (January–June 2026). Each phase includes concrete deliverables and acceptance criteria.

2.2.1 Phase 1 – PREPD deliverable (November 2025–January 2026)

- **Literature review:** Conduct a systematic review using PRISMA to identify observability frameworks and deployment patterns relevant to nonprivileged containers.
- **Technology assessment:** Evaluate candidate technologies and produce a comparative matrix aligned with constraints.
- **Initial architecture design:** Produce a highlevel reference architecture and validate it against constraints.
- **Baseline data collection:** Establish baseline telemetry measurements and measurement procedures.
- **PREPD delivery:** Submit the PREPD report and presentation materials.

2.2.2 Work Plan (extended abstract)

The extended abstract summarises the planned activities and deliverables for the PREPD and thesis phases. It frames Phase 1 as focusing on literature review, technology assessment and baseline data collection, and Phase 2 as addressing prototype implementation, production deployment and evaluation. The present document adopts that plan and expands it with acceptance criteria and measurable outputs.

The main tasks from the extended abstract are reiterated here for traceability:

- **Phase 1 (Nov–Jan):** Systematic literature review using PRISMA; comparative technology assessment; initial architecture design; baseline telemetry collection; PREPD report submission.
- **Phase 2 (Jan–Jun):** Finalise state of the art and select technologies (Jan–Feb); implement prototype and deployment automation (Feb–Mar); coordinate production deployment subject to approvals (Apr); collect observability data and run measurement campaigns (May); validate, analyse and write up results (Jun); final submission (mid-Jun).

Each task includes acceptance criteria such as documented evidence tables for the review, a completed technology assessment matrix, reproducible deployment scripts for the prototype, and an archived dataset with analysis scripts for the evaluation phase.

2.2.3 Phase 2 – Thesis development (January–June 2026)

- **Detailed architecture and prototype implementation (Feb–Mar):** Implement the prototype in staging with reproducible deployment artifacts.
- **Production deployment (Apr):** Coordinate deployment to target servers, following changemanagement procedures.
- **Observability data collection (May):** Execute the measurement campaign and collect quantitative metrics.
- **Validation, analysis and writing (Jun):** Analyze results, collect practitioner feedback and complete thesis chapters.
- **Final submission:** Incorporate feedback and submit the dissertation.

2.2.4 Project Charter, Scope and Schedule

Detailed work breakdowns (WBS), acceptance criteria and the integrated schedule are maintained in project artefacts and updated at milestone reviews.

Key deliverables and acceptance criteria (examples):

- **Deliverable: Reference architecture diagram.** Acceptance: diagram includes component placement, data flows and privilege assumptions, and is reviewed by advisor.
- **Deliverable: Prototype artifacts.** Acceptance: reproducible deployment scripts and container images that can be deployed in a staging environment following the provided README.
- **Deliverable: Evaluation dataset and analysis.** Acceptance: raw telemetry and analysis scripts are archived and sufficient to reproduce primary metrics reported in the thesis.

2.2.5 Monitoring and Controlling Procedures

Progress monitoring: Regular meetings with the advisor and company supervisor and periodic written reports will track milestones and risks.

Risk management: Key risks include infrastructure provisioning delays, limited production access and tooling incompatibilities. Mitigations include extended staging tests, alternative evaluation scenarios using staging data, and early proof-of-concept experiments.

2.2.6 Risk Identification and Management

Risks will be logged in the project risk register, prioritised using an impact/probability matrix, and assigned mitigations and owners. The register will be reviewed at milestone checkpoints and adapted as new information arises.

An illustrative risk table will be maintained in project artefacts; examples of mitigations include the use of staging testbeds to emulate production behaviour and fallback measurement strategies if production deployment is delayed.

Chapter 3

Literature review

3.1 PRISMA Methodology Overview

A systematic literature review will be conducted following Preferred Reporting Items for Systematic Reviews and Meta-Analyses (PRISMA) guidelines to ensure comprehensive and reproducible coverage of the research domain. The search strategy encompasses the following elements:

Information sources:

- **Academic databases:** IEEE Xplore, ACM Digital Library, Springer, ScienceDirect, and Google Scholar for peer-reviewed publications
- **Technical repositories:** arXiv for preprints and working papers in systems and software engineering
- **Gray literature:** Technical documentation from established open-source observability projects, industry whitepapers, and practitioner reports from reputable sources (given the rapid evolution of container observability practices)

Search terms and queries: Core search queries will combine terms from three concept groups using Boolean operators:

1. *Observability concepts:* “observability”, “monitoring”, “telemetry”, “instrumentation”, “metrics”, “logging”, “tracing”, “distributed tracing”
2. *Container and deployment context:* “container”, “containerized”, “Docker”, “non-privileged”, “unprivileged”, “sidecar”, “agent”
3. *Application domain:* “CI/CD”, “continuous integration”, “continuous deployment”, “DevOps”, “reliability”, “incident response”, “MTTD”, “MTTR”

Example query structure: (*observability* OR *monitoring* OR *telemetry*) AND (*container* OR *containerized* OR *non-privileged*) AND (*CI/CD* OR *reliability* OR *incident response*)

Inclusion and exclusion criteria:

- **Inclusion:** Peer-reviewed publications from 2018–2025, technical reports and documentation addressing container telemetry collection, instrumentation patterns, performance evaluation, or reliability improvement in constrained or heterogeneous environments, comparative studies of observability approaches, and empirical evaluations of monitoring overhead

- **Exclusion:** Publications exclusively focused on privileged host access patterns without discussion of constraints, cloud-provider-specific solutions that cannot generalize to on-premise infrastructure, purely theoretical work without practical implementation guidance, and papers predating 2018 (unless seminal works establishing foundational concepts)

Selection process: Literature selection will follow a three-stage process: (1) title and abstract screening to identify potentially relevant sources, (2) full-text review to assess relevance against inclusion/exclusion criteria, and (3) quality assessment focusing on methodological rigor, reproducibility, and applicability to constrained environments. Reference chaining (backward and forward citation tracking) will supplement database searches to identify additional relevant work.

3.1.1 Identification

The identification stage will record search queries, databases queried, and the number of retrieved records. Queries will be versioned and documented to enable replication of the search process.

3.1.2 Screening

Title and abstract screening will be performed using documented inclusion/exclusion criteria; screening decisions and reasons for exclusion at this stage will be recorded. When uncertain, items will be advanced to fulltext review to avoid premature exclusion.

3.1.3 Eligibility

Fulltext review assesses whether the study meets the inclusion criteria and whether it reports sufficient methodological detail for quality assessment. Studies that lack sufficient detail will be noted but may be excluded from quantitative synthesis.

3.1.4 Inclusion and Data Extraction

Included studies will have data extracted into structured evidence tables. Extraction fields include: study context, deployment environment (cloud/onpremise), privilege assumptions, telemetry modalities used, evaluation metrics and experimental design. Extraction will be doublechecked for accuracy.

3.2 Prisma Flow Diagram

The PRISMA flow diagram will be generated from the recorded counts at each stage (identification, screening, eligibility, inclusion) and included in the thesis as a visual summary of the search process.

3.2.1 Quality assessment criteria

Quality assessment will evaluate internal validity (study design, controls), external validity (generalizability to constrained CI/CD environments), and reporting transparency (availability of artifacts, measurement procedures). Assessment outcomes will guide whether a study contributes to quantitative synthesis or only qualitative insight.

3.3 Related Work on Observability in CI/CD Environments

The literature review will examine existing approaches to container observability, deployment patterns for telemetry collection (agent-based, sidecar, host-level), instrumentation frameworks and their privilege requirements, performance and overhead studies, and empirical evaluations of monitoring interventions.

Preliminary reading indicates that much of the documented practice and empirical evaluation in the literature assumes elevated privileges, homogeneous cloud environments and a high degree of control over service code and deployment (for example, the use of sidecar patterns and Kubernetes operators). There is comparatively less empirical guidance for internal, on-premise CI/CD deployments where host administration is restricted and thirdparty packaged services cannot be modified.

Below we summarise representative studies and identify areas where evidence is limited or absent. These summaries are intended to guide the systematic review rather than to be exhaustive.

3.3.1 Representative studies

Pappula et al. (2021) [3] propose a multisignal observability pipeline that integrates OpenTelemetry, Prometheus and centralized tracing to improve fault detection and diagnosis. The study reports reductions in detection and repair times in microservices contexts, and discusses practical considerations such as sampling and storage tradeoffs. Siddiqui et al. (2023) [4] present a Jenkinscentric monitoring approach using Prometheus and Grafana, with concrete KPIs and dashboard templates for pipeline observability. Both studies are informative for architecture and KPI selection but assume administrative access or plugin installation capability.

Each representative study will be summarised in an evidence table capturing scope, methods, results, and limitations. These tables will support crossstudy comparisons and feed into the gap analysis.

3.3.2 Preliminary findings from the extended abstract

The extended abstract contains preliminary synthesis of prior work and initial observations that inform the prototype design. In particular, it highlights two representative studies that shaped the initial selection of KPIs and architectural options: a multisignal pipeline integrating OpenTelemetry, Prometheus and tracing backends [3], and a Jenkinsfocused Prometheus+Grafana monitoring approach [4]. These preliminary findings motivate the selection of MTTD and MTTR as primary outcome measures, and support exploring sidecar and collector deployment patterns that do not require host administrative privileges.

The systematic review will test these preliminary inferences by extracting comparable metrics and contexts from the literature and by documenting differences in how studies define and measure outcomes such as MTTD and MTTR.

3.3.3 Observability frameworks and telemetry collection

The review will compare architectural families (push vs pull, agent vs sidecar, collector topologies) and practical collection techniques (metrics exposition, log forwarding, trace

context propagation). For each family, the review will document privilege assumptions and crossplatform applicability.

3.3.4 Correlation mechanisms

Correlation across signals (metrics, logs, traces) typically relies on propagated identifiers and consistent instrumentation. The review will identify correlation patterns that can operate without modifying application code, for example by leveraging request identifiers available in network or proxy layers, or by enriching logs via sidecar processes.

3.3.5 Performance metrics and evaluation methods

The systematic review will identify common evaluation metrics (MTTD, MTTR, throughput, overhead) and experimental methods (controlled fault injection, A/B comparisons, longitudinal collection), and note measurement pitfalls and reporting gaps.

When extracting metrics, the review will record definitions and measurement procedures used by each study (for example, how MTTD and MTTR are operationalised) to ensure that subsequent comparisons use compatible definitions or explicitly document differences.

3.4 Gaps in Existing Research

Early indications suggest the following gaps: (1) scarce empirical studies focusing on non-privileged, onpremise CI/CD environments; (2) limited guidance for instrumenting thirdparty packaged services without code changes; and (3) few reproducible artifacts (templates, scripts) targeted at constrained deployments. The systematic review will verify and refine this preliminary gap analysis.

3.5 Summary

This chapter establishes the review protocol and situates the initial findings that motivate the design and evaluation phases. The systematic review results will inform technology selection and prototype design in subsequent chapters.

3.6 Answering the Research Question

The systematic review will produce an evidence base to answer the research questions by: (a) identifying feasible telemetry collection patterns under privilege constraints; (b) documenting evaluation methods and KPIs suitable for constrained environments; and (c) providing references and examples to inform the prototype implementation and measurement strategy.

Chapter 4

Conclusion and Next Steps

This chapter summarises the dissertation scope and outlines the immediate next steps for prototype implementation and empirical evaluation. The work presented so far defines the problem context, research objectives, methodology and the protocol for a PRISMA-guided literature review. The subsequent phases will apply the evidence collected in Chapter 3 to select technologies, design a reproducible reference architecture and implement a prototype suitable for constrained, nonprivileged CI/CD environments.

Next steps:

- Finalise the systematic literature review and produce the Prisma flow diagram and evidence tables.
- Complete the technology assessment matrix and select the candidate stack for prototyping, documenting selection rationale and compatibility with privilege constraints.
- Implement the prototype in a staging environment, produce deployment templates and instrument the selected CI/CD services following the reference architecture.
- Execute the measurement campaign (baseline and postdeployment), including controlled fault injection experiments, and collect quantitative and qualitative data according to the evaluation plan.
- Analyse results, generalise findings where appropriate, and prepare dissertation chapters describing implementation, evaluation and implications for practice.

The project will avoid making definitive claims about generalisability until the prototype evaluation and analysis are complete; all conclusions will be grounded in the empirical evidence and contextualised relative to the reviewed literature. The following deliverables are expected at the end of the evaluation phase:

- A documented reference architecture, including deployment templates and configuration artifacts.
- A reproducible prototype that can be deployed in a staging environment using provided scripts and container images.
- An evaluation report containing raw measurement data, analysis scripts, and a reproducible pipeline to regenerate key figures and tables.
- A practitioner guidance note summarising deployment constraints, tradeoffs and recommended practices for nonprivileged CI/CD observability.

Timelines and responsibilities for these deliverables are recorded in project artefacts and will be reviewed during milestone meetings. After the evaluation, results will be compared with the literature synthesis to position the contributions and limitations of the work.

Potential dissemination channels include the thesis, technical reports, and possible conference or workshop submissions focusing on observability in constrained environments; decisions on dissemination will observe any organisational publication policies regarding operational data.

4.0.1 Validation targets from the extended abstract

The extended abstract proposes a focused set of evaluation targets that will be used to validate the prototype. These include:

- Primary metrics: reductions in MTDD and MTTR compared to baseline measurements.
- Pipeline throughput: verification that the telemetry pipeline can sustain the required ingestion rate without data loss under expected load profiles.
- Alert precision and detection coverage: measured via controlled fault injection and posthoc analysis of alerting outcomes.
- Instrumentation overhead: quantification of CPU, memory and network overhead introduced by the observability stack, with an operational target of keeping overhead below 5% where feasible.

These targets will be operationalised with specific measurement procedures and acceptance criteria documented in the evaluation plan; conclusions will be restricted to the contexts and environments tested and will not be overgeneralised without supporting evidence.

Bibliography

- [1] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.
- [2] J. Volz, "Prometheus: Monitoring at soundcloud," *SoundCloud Developers Blog*, 2016.
- [3] K. K. Pappula, S. Anasuri, and G. P. Rusum, "Building observability into full-stack systems: Metrics that matter," *International Journal of Emerging Research in Engineering and Technology*, vol. 2, no. 4, pp. 48–58, 2021.
- [4] I. Siddiqui, A. Pandey, S. Jain, H. Kothadia, R. Agrawal, and N. Chankhore, "Comprehensive monitoring and observability with jenkins and grafana: A review of integration strategies, best practices, and emerging trends," in *2023 7th International Symposium on Multidisciplinary Studies and Innovative Technologies (ISMSIT)*, Oct. 2023, pp. 1–5. DOI: 10.1109/ISMSIT58785.2023.10304904.

Appendix A

Appendix Title Here

Write your Appendix content here.